EF Core Aggregations

Count() • Sum() • Average() • Min() • Max() | Combined lesson summary

Core idea

Aggregation methods calculate a summary from a set of rows. When used on an EF Core database query, the aggregate can be translated to SQL so SQL Server performs the calculation and the application receives a scalar result.

Mental anchor: Filter if needed → aggregate in the database → receive the result, not every row.

The five aggregate methods

Method	Purpose	SQL function	Example use
Count()	Counts matching rows.	`COUNT(*)`	Number of active students.
Sum()	Adds numeric values.	`SUM(column)`	Total course duration.
Average()	Calculates the arithmetic mean.	`AVG(column)`	Average enrollment progress.
Min()	Returns the smallest value.	`MIN(column)`	Cheapest course / earliest date.
Max()	Returns the largest value.	`MAX(column)`	Highest price / latest date.

Database-side examples

AggregationExamples.cs — C# • EF Core

```
1  int activeCount = context.Students
2  .Count(s => s.Status == "Active");
3
4  decimal averageProgress = context.Enrollments
5  .Average(e => e.ProgressPercent);
6
7  int totalHours = context.Courses
8  .Sum(c => c.DurationHours);
9
10 decimal lowestPrice = context.Courses.Min(c => c.Price);
11 decimal highestPrice = context.Courses.Max(c => c.Price);
```

These aggregate terminal methods execute the query. EF Core translates supported operations to SQL Server aggregate functions.

Why ToList() before aggregation is usually inefficient

Less efficient	Preferred
Load many rows into application memory, then calculate in C#.	Let SQL Server calculate the aggregate and return one value.
More transferred data, memory use, and client-side work.	Less network traffic and typically better performance.

AvoidThis.cs C# • EF Core

```
var prices = context.Courses
.Select(c => c.Price).ToList();
decimal maxPrice = prices.Max();

// Better: aggregate in the database
decimal maxPrice = context.Courses.Max(c => c.Price);
```

ToList() executes and materializes the query. Avoid it when you only need an aggregate value.

Filtering before aggregation

Combine Where() with an aggregate when the result should cover only matching rows. For example, find the highest price among published courses:

FilteredAggregate.cs C# • EF Core

```
decimal maxActiveCoursePrice = context.Courses
.Where(c => c.Status == "Published")
.Max(c => c.Price);
```

The filter is applied by the database as part of the query.

Min() / Max() versus OrderBy()

If you only need the lowest or highest **value**, prefer Min() or Max(). Sorting and taking First() can be appropriate when you need the entire entity or row associated with that value.

MinMaxVsOrderBy.cs C# • EF Core

```
// Only the lowest price is needed
decimal cheapestPrice = context.Courses.Min(c => c.Price);

// Use ordering when you need the course record itself
var cheapestCourse = context.Courses
.OrderBy(c => c.Price)
.FirstOrDefault();
```

SQL preview versus executed SQL

ToQueryString() previews the SQL shape for an IQueryable query; it does not execute that query. When called before Count(), Average(), Sum(), Min(), or Max(), it may show the underlying selection query rather than the final aggregate SQL. Runtime logging (for example, LogTo) shows the SQL actually executed by the aggregate method.

TypicalAggregateSql.sql C# • EF Core

```
SELECT COUNT(*) FROM [Students] AS [s]
WHERE [s].[Status] = N'Active';

SELECT AVG([e].[ProgressPercent])
FROM [Enrollments] AS [e];

SELECT SUM([c].[DurationHours])
FROM [Courses] AS [c];

SELECT MIN([c].[Price]), MAX([c].[Price])
FROM [Courses] AS [c];
```

Practical notes and common pitfalls

• **Empty results:** Average(), Min(), and Max() over an empty sequence may throw for non-nullable result types. Handle the possibility of no matching rows when relevant.
• **Nulls:** SQL aggregate functions generally ignore NULL values; exact behavior and result handling depend on the expression and provider.
• **Ordering:** Do not sort the whole set merely to obtain a minimum or maximum scalar value.
• **Logging:** EnableSensitiveDataLogging() may expose sensitive values in logs; use cautiously, especially outside development.

Professional use cases

Dashboards • reports • KPIs • active-student counts • total course hours • average progress • cheapest course • highest salary • earliest/latest dates

> **Remember: Filter first when needed, then aggregate in SQL Server. Use Min()/Max() for extremes and avoid loading rows you do not need.**